Organisation Development Standard

Architecture technique et interface • Évolution 0.3.0

Un environnement de développement gouverné dans VS Code : Starter, IA, Quality, Security et GitFlow. Cette édition remplace les parcours SSO et scans liés à un commit par une connexion Git native et des analyses des sources enregistrées.

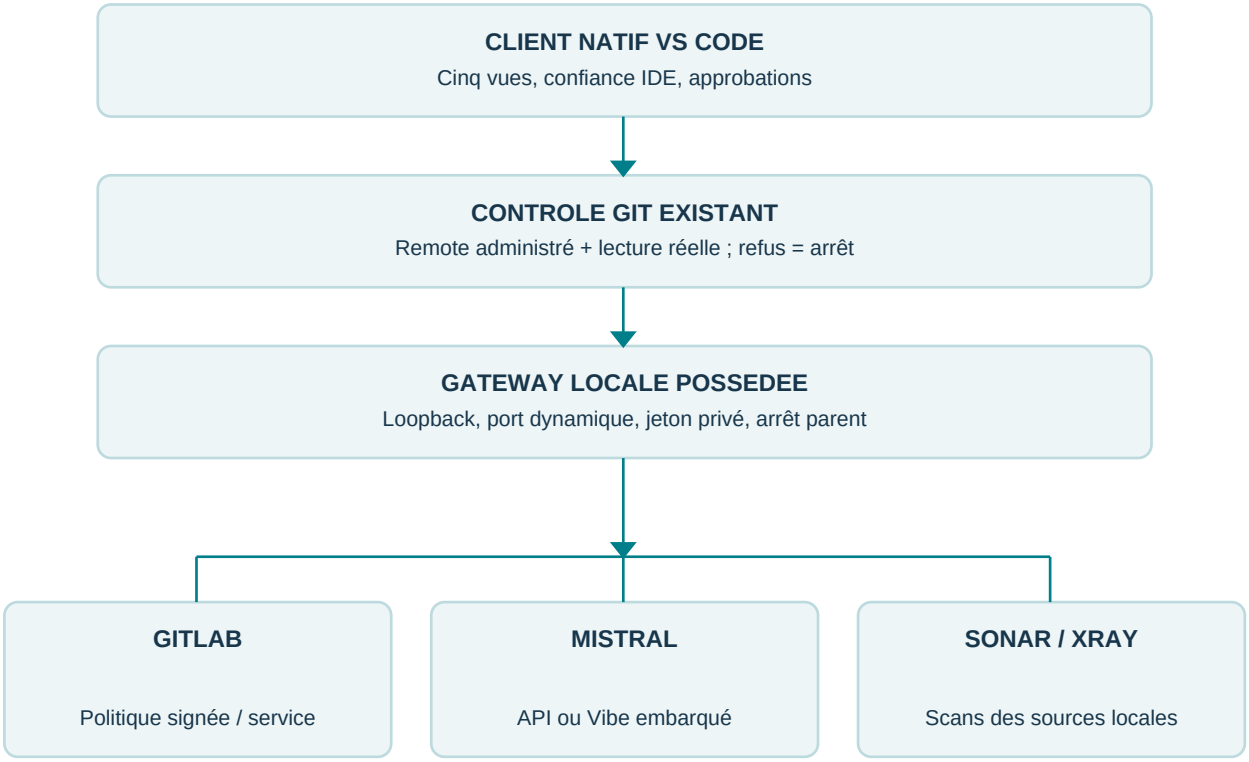
Décision	Comportement cible
Sans SSO	Le développeur utilise l’accès Git déjà disponible dans son IDE. Un refus Git interdit la connexion au gateway.
Configuration au build	L’organisation fournit URL, comptes de service, transport Mistral, watch Xray et mappings Sonar. L’interface utilisateur ne les modifie pas.
Analyses locales	SBOM CycloneDX, Graph Scan Xray et scanner Sonar fonctionnent sans attendre un commit.
Cycle de vie	Le gateway embarqué démarre après le contrôle Git et s’arrête avec l’extension ou la fermeture du canal parent.

Limite acceptée : les secrets distribués dans un VSIX autonome restent extractibles par le propriétaire du poste. Le verrouillage de l’interface, les empreintes et le masquage des logs ne garantissent pas leur confidentialité contre lui.

Référentiel : specification.md, exigences DEVAI-001 à DEVAI-179. Les résultats réellement exécutés sont séparés des objectifs de recette dans docs/VALIDATION.md.

1. Architecture et responsabilités

DEVAI-002, 003, 168, 169



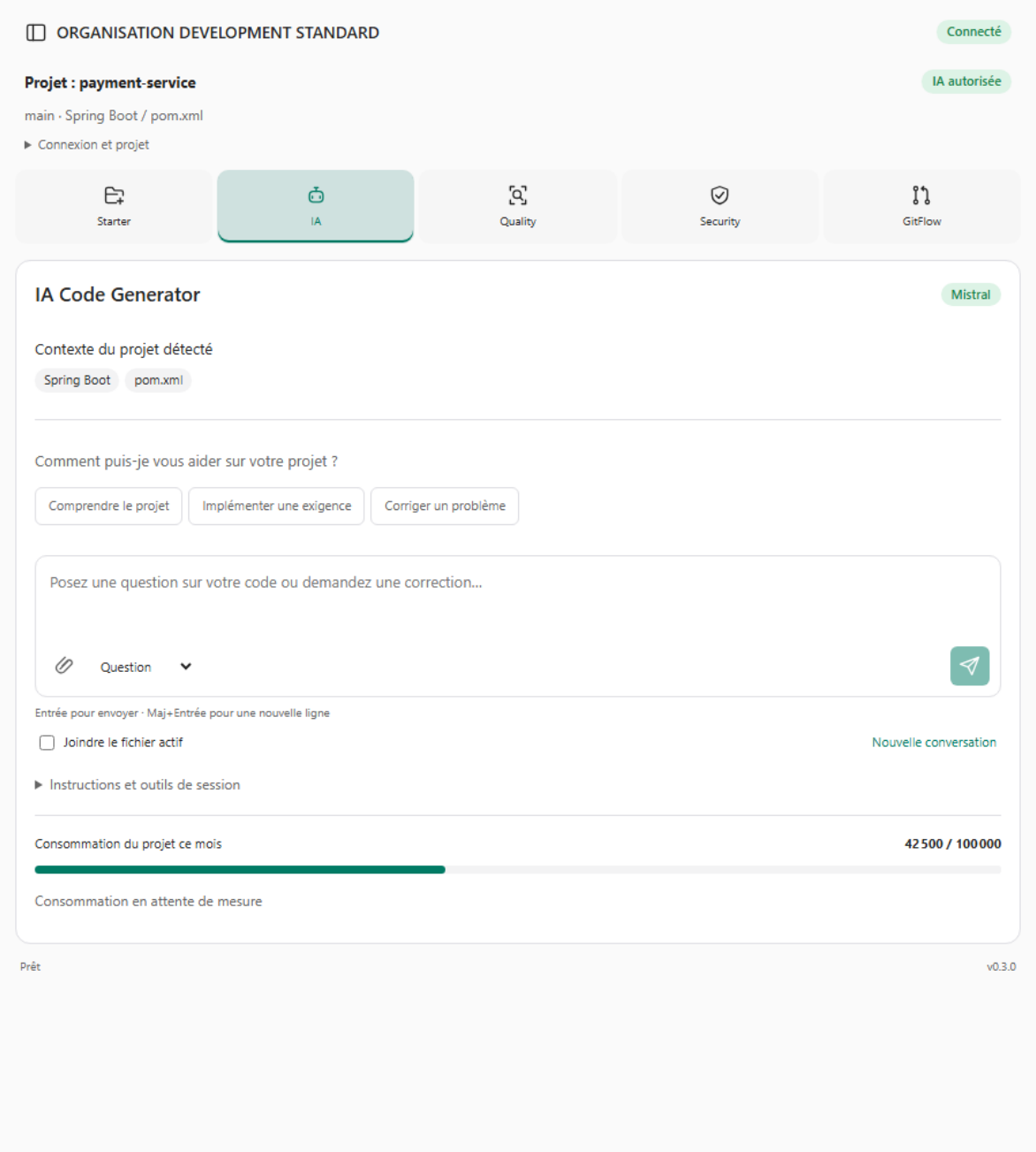
Le moteur Java local conserve les outils de contexte, diff et Git via JSON-RPC sur stdio. Le gateway Spring Boot embarqué exécute les intégrations de service et les décisions IA. Les credentials de publication Git restent ceux du développeur.

Composant	Responsabilité
Client TypeScript	Vérification Git, confiance workspace, vues, permissions et processus possédés.
Moteur Java 21	Lecture filtrée, propositions, checkpoints, commandes approuvées et GitFlow.
Gateway Java 21	Politique signée, Mistral, jobs, quotas locaux, Sonar et Xray.
Adaptateurs Python	Backend Vibe isolé et exécution bornée des scanners de stack.

2. Les cinq onglets

DEVAI-013, 019, 172, 174, 179

L’onglet IA conserve le compositeur, les pièces jointes, les modes Question / Modification / Agent, les instructions et la consommation. Quality et Security portent sur les fichiers enregistrés.



Capture issue des tests de l’interface réelle avec contexte synthétique ; elle ne prouve pas une connexion à un service d’organisation. Aucun secret n’est transmis à la webview.

3. Construire le profil d’organisation

DEVAI-009, 127, 167, 176

Le profil organisation.json déclare l’identité de l’extension, les fichiers env et les fichiers de configuration. Les valeurs environment remplacent celles des envFiles, chargés dans l’ordre. Les références ${VARIABLE_BUILD} sont résolues explicitement au packaging ; une référence manquante bloque la livraison.

Entrée	Usage
extension	name, publisher, displayName : identité et titre du VSIX.
envFiles / environment	URL et credentials des services, mode API ou VIBE_CLI, authentification Xray, watch.
files	Mappings Sonar et ressources administrées copiés dans runtime/config.
Commande	script.bat <PROFIL_ORGANISATION> : validation complète et suiteInstall. generate-vsix.bat : VSIX après validation.
Résultat	Gateway, configuration compilée, scripts et code Vibe intégrés. Sources non réécrites avec les secrets.

Les batch appellent Python, sans PowerShell. Java 21, Node 24, Maven via le wrapper et Python 3.12 sont utilisés au build. Vibe est empaqueté pour la plateforme cible ; le runtime Python reste un prérequis du poste.

Le profil demo contient des placeholders et ne simule aucune connexion métier dans le produit. Les profils privés et artefacts contenant des secrets sont distribués uniquement dans le canal privé autorisé.

4. Connexion Git et durée de vie

DEVAI-020 à 024, 168, 169

Étape	Contrôle effectif
1. Workspace	Le dossier doit être approuvé dans VS Code. Le remote origin doit appartenir au host GitLab administré.
2. Accès Git	git ls-remote utilise le credential helper ou l’agent SSH existant. Aucun compte de service ne remplace cette vérification.
3. Démarrage	Lancement du JAR possédé, port loopback dynamique et jeton aléatoire transmis uniquement au client natif.
4. Projet	Le gateway lie les lectures au chemin du dépôt validé. La politique signée décide séparément les droits IA.
5. Fin	Déconnexion, fermeture de projet, désactivation ou fin du canal parent arrêtent les processus possédés.

Actualiser renouvelle le contrôle Git. Un changement de dépôt invalide la connexion précédente. Les credentials Git du développeur ne sont ni effacés ni remplacés à la déconnexion.

Masquer la vue ne désactive pas l’extension. Les données sont isolées par workspace et chiffrées localement. Deux fenêtres sur le même workspace peuvent rencontrer le verrou de la base ; le produit n’arrête pas l’autre instance.

Le propriétaire du poste peut modifier le programme local. Ce contrôle ne constitue donc pas une preuve d’identité inviolable pour un service central partagé.

5. Mistral API ou Vibe embarqué

DEVAI-063, 064, 170, 171, 176

Choix de build	Exécution
API	Adaptateur HTTP Mistral direct avec ODS_MISTRAL_URL et ODS_MISTRAL_API_KEY.
VIBE_CLI (défaut)	Processus Python isolé utilisant le backend Mistral officiel de la distribution Vibe 3.0.3 embarquée.
API_KEY	Le backend Vibe reçoit la clé API administrée.
CLI_TOKEN	Le backend Vibe reçoit ODS_VIBE_TOKEN, credential accepté par Mistral. Aucun cookie navigateur arbitraire n’est converti en clé.
Modèle	ODS_MISTRAL_MODEL et ODS_MODEL_ALIAS doivent correspondre au modèle autorisé par la politique.

L’adaptateur CLI utilise le code officiel de Vibe mais ne lance pas son interface interactive. Les outils de terminal, MCP et hooks personnels Vibe ne sont pas activés. Les messages, appels d’outils et usages sont normalisés dans le contrat ODS.

Cette exécution conserve les diff, approbations et outils typés ODS. Le processus utilise un répertoire temporaire et n’importe pas la configuration personnelle Vibe du développeur. Les credentials arrivent sur stdin privé, pas dans les arguments.

Le code et les dépendances Vibe sont accompagnés de leurs licences et empreintes. La qualification avec une clé réelle, le modèle et l’offre Mistral de l’organisation reste distincte du test avec transport simulé.

6. Security : générer le SBOM

DEVAI-095, 172, 175

Générer SBOM fonctionne à la demande, sans commit. La sélection de stack est native si plusieurs stacks sont détectées. Les fichiers modifiés dans l'éditeur doivent être enregistrés avant l'analyse. Le générateur produit un vrai document CycloneDX JSON exportable.

Stack	Générateur et entrées
Java	Plugin Maven CycloneDX 2.9.1, makeAggregateBom sur pom.xml.
Angular	CycloneDX npm 6.0.1 via npx, package-lock.json requis pour les versions résolues.
Python	cyclonedx-py : requirements épinglés, Poetry lock ou Pipenv lock.
.NET	dotnet-CycloneDX compatible avec --spec-version, sur une solution ou un projet explicite.

Télécharger SBOM utilise la boîte de sauvegarde native. Le dossier proposé est hors du dépôt pour éviter de modifier son empreinte. Le même SBOM peut ensuite être transmis à Xray. Une modification des entrées exige une nouvelle génération.

Les outils de stack sont des prérequis : leur absence, une erreur de build ou un document invalide ne produit pas de faux rapport. Les commandes susceptibles d'exécuter le projet demandent une approbation native. Les sorties brutes potentiellement sensibles ne sont pas répercutées dans les logs de l'extension.

7. Security : Graph Scan Xray

DEVAI-096 à 105, 173, 176

Le protocole reprend l'appel du projet scan-sbom-xray. Le SBOM est converti en graphe de composants component_id / nodes. Les identités Maven, npm avec scope, PyPI et NuGet sont normalisées. Les cycles sont bornés et les composants déconnectés sont conservés.

Requête	Contrat
POST	/xray/api/v1/scan/graph?watch=...&scan_type=dependency ; projectKey facultatif.
Réponse	scan_id obligatoire, puis interrogation du résultat.
GET	/xray/api/v1/scan/graph/{scan_id}?include_vulnerabilities=true&include_licenses=true
202	Analyse en cours : attente bornée, annulation contrôlée, aucun résultat final inventé.
200	Rapport avec vulnérabilités et/ou violations, CVE, composants, versions corrigées et chemins d'impact.

BASIC : username et mot de passe du compte de service. TOKEN : Bearer token JFrog, avec username associé dans le profil administrateur. URL, mode et watch sont figés au build. Le développeur ne peut pas les éditer dans Security.

Corriger les issues transmet uniquement les constats sélectionnés au LLM gouverné. Une version corrigée proposée par le rapport reste à vérifier contre les contraintes du projet et un nouvel audit réel.

8. Quality : analyser avant commit

DEVAI-084 à 094, 174, 175

Le bouton Analyser remplace Analyser le commit. L'analyse porte sur les fichiers locaux enregistrés, sans commit ni push automatique. Le scanner reçoit uniquement son token Sonar et la destination du profil.

Stack	Parcours scanner
Java	mvn verify puis plugin Sonar Maven sur le même workspace.
Angular / Python	SonarScanner CLI installé et configuration du projet.
.NET	SonarScanner begin, dotnet build --no-incremental, end. Paramètres sensibles via SONARQUBE_SCANNER_PARAMS.
Provenance	report-task.txt produit par le scanner, ceTaskId, attente de la tâche CE puis analysisId.
Rapport	Issues paginées et quality gate de l'analyse soumise ; refus si l'analyse courante change pendant la lecture.

L'organisation fournit l'URL, le token de service disposant du droit d'analyse, son nom de compte et le mapping entre ID GitLab et clé Sonar. Le token authentifie les appels ; le username est le libellé du compte associé.

Ce parcours est une analyse à la demande. Il ne promet pas l'exécution continue à chaque frappe de SonarQube for IDE. Les versions du scanner, du serveur et les licences doivent être qualifiées ensemble par l'organisation.

9. IA : envoi et remédiation

DEVAI-068, 090, 098, 151 à 160, 175, 179

Entrée envoie une seule demande non vide lorsque l'IA est autorisée. Maj+Entrée ajoute une ligne ; une composition IME ne soumet pas. Le bouton Envoyer suit le même contrôleur. Le brouillon est conservé après refus et effacé après acceptation du job.

Remédiation	Condition
Sélection	Choisir les issues du rapport Sonar ou Xray, puis le mode Modification ou Agent.
Fraîcheur	L'empreinte actuelle doit être égale à celle du rapport ; sinon une nouvelle analyse est exigée.
Autorisation IA	La politique signée doit autoriser la tâche de remédiation pour le projet et le modèle.
Traitement	Le rapport est une donnée non fiable, jamais une instruction privilégiée. Inspecter les sources pertinentes.
Validation	Diff relu, modifications acceptées, builds et tests autorisés, puis nouveau scan avant d'annoncer la correction.

L'agent conserve ses outils typés, permissions de session, checkpoints et annulation. Une nouvelle intention ou un changement de contexte invalide les permissions concernées. Aucun commit ou push ne résulte implicitement d'une correction.

Les tests traversent le compositeur réel, le contrôleur natif et le client HTTP jusqu'à une réponse LLM simulée. Ils n'utilisent pas les credentials du poste.

10. Gouvernance et secrets

DEVAI-026 à 044, 161 à 167, 176, 178

Le compte de lecture GitLab et son token, fournis au build, lisent la politique du dépôt privé. La référence protégée est résolue en commit ; les octets sont vérifiés par signature Ed25519. Les droits IA sont distincts pour chaque action et refusés par défaut.

Protection	Portée réelle
Interface	Aucun éditeur de service, secret ou URL dans les onglets ; seules des informations publiques sont affichées.
Logs	Masquage des valeurs connues et formes d'authentification ; pas de corps d'erreur fournisseur ni de sorties brutes de scanner.
Processus	Pas de credentials Mistral, Xray ou gouvernance transmis aux builds de projet. Le scanner Sonar reçoit son seul credential nécessaire.
Distribution	Secrets extractibles dans le VSIX autonome. Rotation par rebuild privé et réinstallation.
Commentaires	Chaque symbole ODS porte une explication et les IDs d'exigences pertinents. Les vérificateurs contrôlent aussi les IDs connus.

Le code tiers Vibe conserve ses licences et commentaires originaux ; aucun numéro d'exigence ODS n'est attribué artificiellement à ses auteurs. La revue de pertinence des commentaires complète leur vérification syntaxique.

11. Recette et livraison

DEVAI-133 à 150, 177

Niveau	Vérification attendue
Java	Compilation Maven complète, tests de contrats et de services, refus d’accès et annulation.
Python	Profils, résolution de secrets, quatre générateurs SBOM, provenance et credentials des scanners.
Vibe	Code embarqué et SDK réels avec frontière HTTP simulée ; requête, clé, modèle, réponse et usage vérifiés.
Interface	Tests Node et Chromium, Enter / IME / Maj+Entrée, boutons locaux, conservation du brouillon et rendu étroit.
Artefact	VSIX inspecté, gateway possédé démarré et arrêté, compatibilité VS Code 1.95.2, version et empreintes de livraison.

Tous les services métier sortants sont remplacés par des doubles explicites dans les tests. Le téléchargement des dépendances de build n’est pas une qualification d’un service métier. Les preuves réellement exécutées et les éventuels échecs figurent dans docs/VALIDATION.md.

suiteInstall contient les binaires du build courant, configuration exemple, scripts et documentation. Il reste un artefact hors Git. Une release exige la correspondance version / commit / tag / package et ne doit pas être annoncée avant publication vérifiée.

Les tests réels avec les comptes, modèles, permissions, scanners et licences de l’organisation restent une qualification séparée. Un mock réussi ne remplace pas cette validation.

12. Traçabilité de l’évolution

Référentiel de mise en œuvre et de revue

Exigence	Évolution
DEVAI-167	Configuration et secrets administrés au build.
DEVAI-168	Gateway embarqué, jeton privé et durée de vie possédée.
DEVAI-169	Accès Git existant avant connexion, aucun SSO dans le VSIX.
DEVAI-170 / 171	Mistral API ou Vibe, code et dépendances embarqués.
DEVAI-172 / 173	SBOM quatre stacks et Graph Scan Xray avec watch.
DEVAI-174 / 175	Qualité locale, provenance et correction liée au rapport.
DEVAI-176	Masquage, isolation des credentials et limite d’extractabilité.
DEVAI-177	Doubles de toutes les intégrations métier.
DEVAI-178	Commentaires explicatifs et numéros d’exigence vérifiés.
DEVAI-179	Envoi effectif du compositeur jusqu’au transport LLM simulé.

Documents associés : specification.md ; prompt-developpement.md ; docs/CONFIGURATION.md ; docs/INSTALLATION.md ; docs/API.md ; docs/VALIDATION.md. Le prompt demande d’examiner le code existant, de maintenir les commentaires, d’implémenter les parcours complets et de distinguer les validations exécutées des éléments non vérifiables.

Les capacités du serveur séparé historique sont conservées pour non-régression. Son authentification OIDC et ses runners ne constituent pas un prérequis du parcours autonome décrit ici.